Assignment 9

Task 1 Compiling Expressions

Using the attributed grammar described on lecture slides 40-51, determine the sequence of MJVMUsing the attributed grammar described on lecture slides 40-51, determine the sequence of MJVM instructions generated for the expression below. Moreover, represent the concrete syntax tree and operand descriptors as on lecture slide 52, and explain how operand descriptors are used to generate the instruction sequence during parsing.

```
(P * a[i] + x) % 31
```

a is a local variable at address 1 pointing to an integer array, i is a local variable at address 0, x is a global variable at address 9, and P is a constant with value 11.

Operand Descriptors

We first describe the operands involved in the expression:

Part	Kind	Value/Address
`a`	`Local`	1
`i`	`Local`	0
`x`	`Static`	9
`P`	`Con`	11
`a[i]`	`Elem`	—
`31`	`Con`	31

Syntax Tree

```
Expr
└─ Term (%)
   ├─ Expr (+)
   │  ├─ Term (*)
   │  │  ├─ Factor: P = 11 (Con)
   │  │  └─ Factor: a[i] (Elem)
   │  └─ Factor: x (Static)
   └─ Factor: 31 (Con)
```

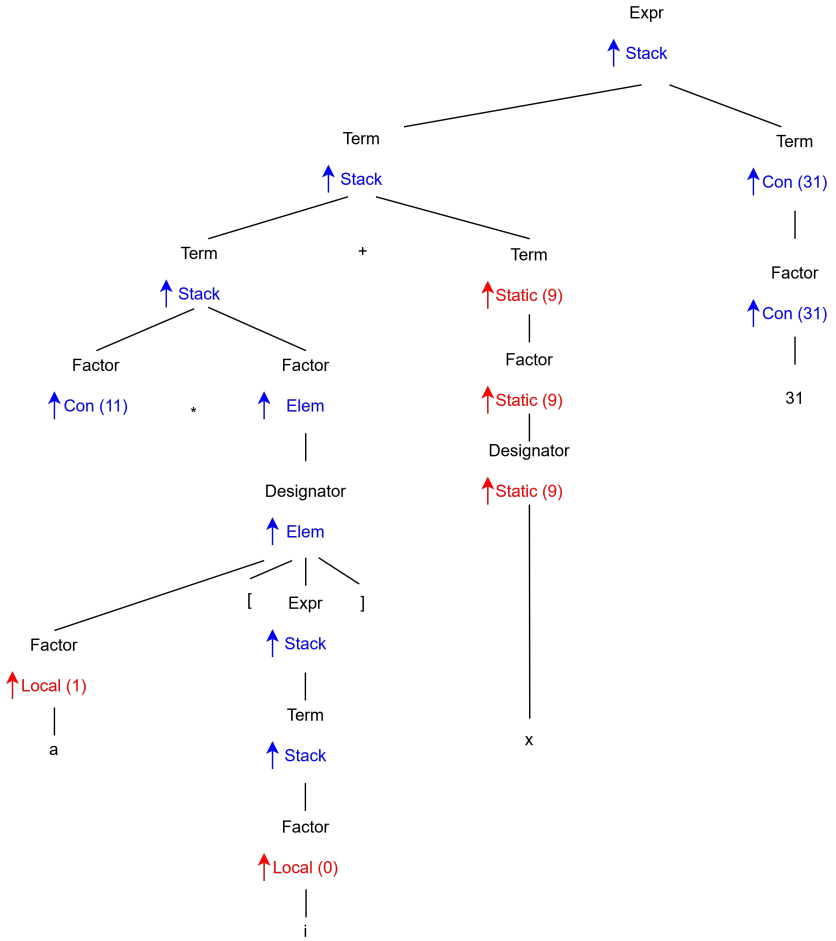
Bytecode Instruction Sequence

Step	Instruction	Explanation	EStack After
1	`const 11`	Load constant P (value 11) onto the EStack	11
2	`load1`	Load base address of local array variable 'a' (address 1)	11, a
3	`load0`	Load index from local variable 'i' (address 0)	11, a, i
4	`aload`	Load integer element a[i] onto the EStack	11, a[i]
5	`mul`	Multiply 11 * a[i]	result1
6	`getstatic 9`	Load global variable 'x' from data address 9	result1, x
7	`add`	Add Add (P * a[i]) and x	result2
8	`const 31`	Load constant 31 onto the EStack	result2, 31
9	`rem`	Compute remainder of the division by 31	final result

Address	Instruction	Bytes	Estack
0	const 11	5	11
5	load1	1	11, adr(a)
6	load0	1	11, adr(a), i
7	aload	1	11, a[i]
8	mul	1	(P * a[i])
9	getstatic 9	3	(P * a[i]), x
12	add	1	(P * a[i] + x)
13	const 31	5	(P * a[i] + x), 31
18	rem	1	((P * a[i] + x) % 31)
19			

Concrete Syntax Tree and Operand Descriptors

The compilation process utilizing operand descriptors as intermediate syntax attributes can be visualized via the following evaluation hierarchy:



Task 2 MJ Compiler and Decoder

Use the given expression of Task 1 in a complete (minimal) MJ source program and compile it using MJ.Compiler (see lecture slide 40 of the "Overview" chapter). Decode the generated object file using MJ.Decode, then locate and explain the generated code for the expression of Task 1. Addresses of variables may differ from Task 1

1. Theoretical Background

In this task, the goal is to compile and decode the MJVM instructions corresponding to the expression:

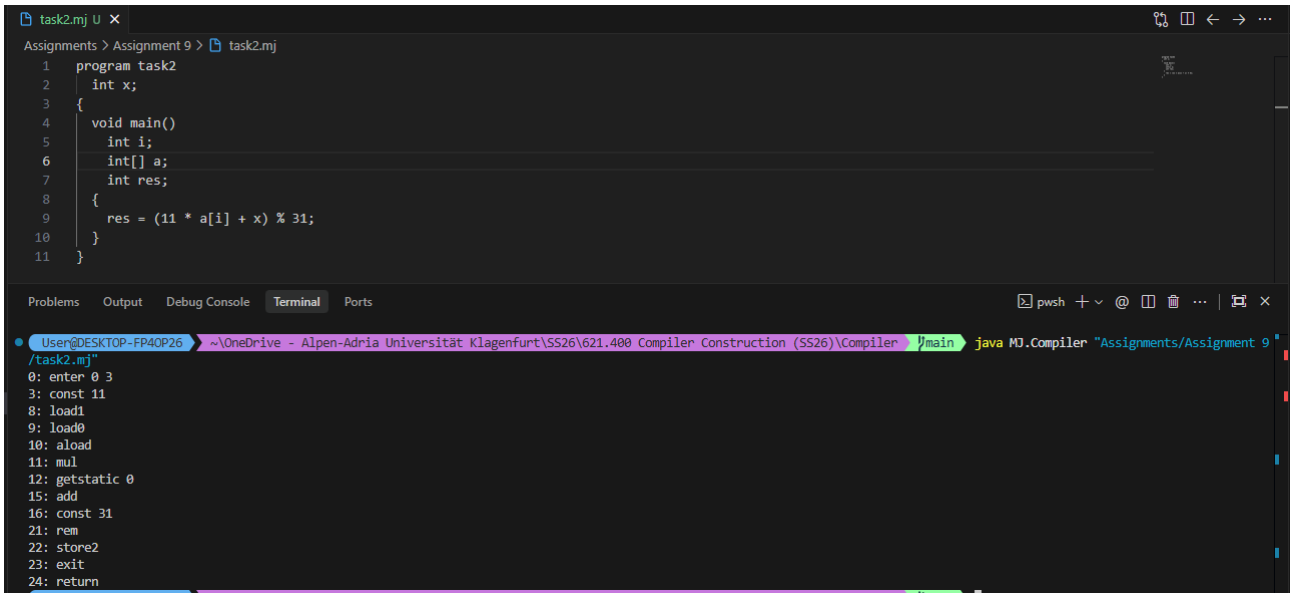
```
(P * a[i] + x) % 31
```

Compile and Decode test program Test.mj (https://ssw.jku.at/CompilerBuch/)

```
java MJ.Compiler "Assignments/Assignment 9/task2.mj"
java MJ.Decode "Assignments/Assignment 9/task2.mj"
```

Program

```
program task2
  int x;
{
  void main()
    int i;
    int[] a;
    int res;
  {
    res = (11 * a[i] + x) % 31;
  }
}
```



Task 3 Compiling Assignments

Using the attributed grammar described on lecture slides 40-56, determine the sequence of MJVM instructions generated for the assignment sequence below. Moreover, for each generated instruction, give the production and semantic action (method call of Code class) of the attributed grammar that emits the instruction. Also reference the lecture slide number where each semantic action is visible.

```
n = 13;
obj = new Dataset;
obj.val = new int[20];
obj.val[0] = n;
```

n is a local variable at address 1, obj is a global variable at address 3, Dataset is a class with 2 fields, where val is the second field. All variables and fields have been declared such that the given assignments are valid.

Compilation Mapping Table

The following table outlines the complete sequence of generated MJVM instructions for the given assignment sequence, along with their corresponding production context, semantic actions in the `Code` class, and the lecture slide numbers:

Assignment Statement	MJVM Instruction	Semantic Action / Method Call	Slide
`n = 13;`	`const 13`	`Code.load(y)` (via Expr)	37
	`store1`	`Code.assignTo(x)` (Local slot 1)	56
`obj = new Dataset;`	`new 2`	`Code.put(Code.new_);`	51
		`Code.put2(2);`	
	`putstatic 3`	`Code.assignTo(x)` (Static slot 3)	56
`obj.val = new int[20];`	`getstatic 3`	`Code.load(x)` (Load base object reference)	40
	`const 20`	`Code.load(y)` (Load array size expression)	37
	`newarray 1`	`Code.put(Code.newarray);`	51
		`Code.put(1);`	
	`putfield 1`	`Code.assignTo(x)` (Store to second field)	56
`obj.val[0] = n;`	`getstatic 3`	`Code.load(obj)` (Load base reference)	40
	`getfield 1`	`Code.load(fld)` (Load array field pointer)	37
	`const0`	`Code.load(idx)` (Load implicit constant 0)	37
	`load1`	`Code.load(y)` (Load local value from slot 1)	37
	`astore`	`Code.assignTo(x)` (Array store element)	56

Table 1: Attributed grammar mechanics and code generation mapping.

Structural Step-by-Step Derivation

- **Field Adressing (`val`):** Since `Dataset` contains 2 fields and `val` is explicitly designated as the second field, it maps to the relative 0-indexed offset `1`.
- **Array Handling:** The production for array instantiation (`new int[20]`) enforces an evaluation of the size expression first. Because it is of type `int`, `newarray 1` is emitted (where `1` signifies a word-sized primitive array allocation on the heap).
- **Assignment Synchronization:** For complex left-hand-side designators like `obj.val[0]`, the compiler emits tracking and base-loading bytecode (`getstatic`, `getfield`) immediately during the left-to-right designator parsing phase. The actual storing instruction (`astore`) is generated only at the very end of the statement production via `Code.assignTo()`.

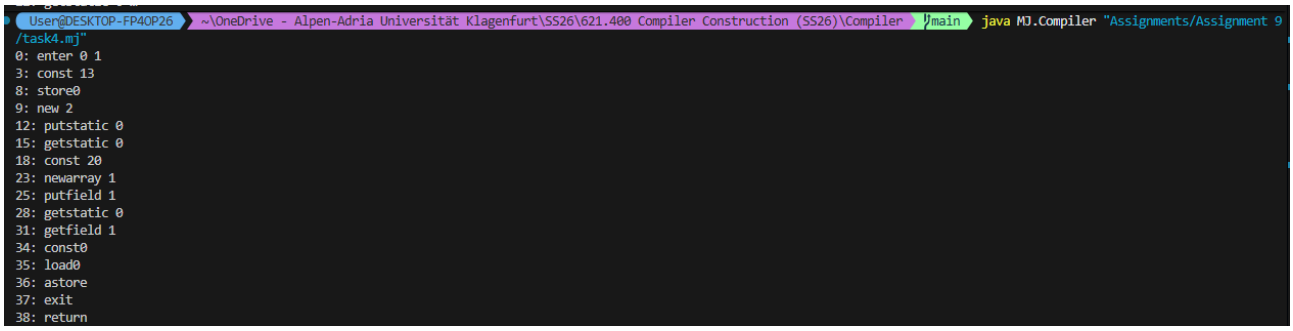
Task 4 MJ compiler and decoder

Same task as Task 2, but for the given assignment sequence of Task 3.

```
program task4
  class Dataset {
    int dummy; // First field (Offset 0)
    int[] val; // Second field (Offset 1)
  }
  Dataset obj; // Global variable at static address 0
{
  void main()
    int n; // Local variable at offset 0
  {
    n = 13;
    obj = new Dataset;
    obj.val = new int[20];
    obj.val[0] = n;
  }
}
```

Comparison

```
0: enter 0 1
3: const 13
8: store0
9: new 2
12: putstatic 0
15: getstatic 0
18: const 20
23: newarray 1
25: putfield 1
28: getstatic 0
31: getfield 1
34: const0
35: load0
36: astore
37: exit
38: return
```



Task 5 Jump labels

The following Java code snippet uses Code and Label classes of the MJ.CodeGen package of the MicroJava compiler to generate code for an if-else statement. x, y, and z are operand descriptors referring to local variables at addresses 0, 1, and 2, respectively. c is an operand descriptor representing a constant with value 29.

```
Label elseLabel = new Label();
Label endLabel = new Label();
Code.load(x); Code.load(y);
Code.put(Code.jlt); elseLabel.putAdr();
Code.load(c); Code.assignTo(z);
Code.put(Code.jmp); endLabel.putAdr();
elseLabel.here();
Code.load(y); Code.assignTo(z);
endLabel.here();
```

Determine the MJVM instructions generated by this code snippet, assuming that the first generated instruction is located at code address 100. For each MJVM instruction, represent its code address, instruction including explicit operands, and instruction length (in bytes). Moreover, explain when the final jump target addresses are written to the code buffer during execution of the given Java code snippet.

1. Generated MJVM Instructions Table

Assuming the first generated instruction is located at code address 100, the following table lists the precise bytecode sequence, explicit operands, instruction lengths, and relative jump target calculations:

Address	Instruction	Opcode (Hex)	Length (Bytes)	Explanation
100	`load0`	0x02	1	Load local variable x from stack frame offset 0.
101	`load1`	0x03	1	Load local variable y from stack frame offset 1.
102	`jlt 12`	0x2D	3	Conditional jump on less-than via opcode 45 to `elseLabel` (Target 114: relative offset $114 - 102 = 12$).
105	`const 29`	0x16	5	Load integer constant 29 via opcode 22 (1-byte opcode + 4-byte big-endian word).
110	`store2`	0x09	1	Store evaluated value into local variable z at offset 2.
111	`jmp 5`	0x2A	3	Unconditional jump via opcode 42 to `endLabel` (Target 116: relative offset $116 - 111 = 5$).
114	`load1`	0x03	1	Logical anchor of `elseLabel.here()`; load variable y.
115	`store2`	0x09	1	Store evaluated value into local variable z at offset 2.
116	`---`	—	—	Logical anchor of `endLabel.here()` points to this immediate next execution slot.

Table 2: MJVM instruction sequence and code array trace for forward jump resolutions.

2. Mechanics of Forward Jump Target Address Patching

Because forward jumps target labels whose absolute locations are not yet reached during compilation, the target address writing inside the byte buffer cannot happen instantly. The implementation in the `Label` class splits this process into two critical steps:

- **Placeholder Allocation (`putAdr()`):** When the parsing framework encounters a jump statement (like `jlt` or `jmp`), it writes the 1-byte opcode into the code buffer. Then, `putAdr()` injects a temporary 2-byte placeholder field (`0`) into the stream and registers the exact stream offset (`Code.pc`) into the label object's internal tracking array list (`fixupList`).

- **Retrospective Patching (`here()`):** As soon as the source program traversal encounters the physical anchor node (such as the `else` branch block start or the closing boundaries of the structure), the corresponding label invokes `here()`. This marks the current `Code.pc` position as the definite destination. The method automatically iterates backwards through all code addresses collected in its `fixupList` and patches the finalized distance bytes via `Code.put2(pos, target - (pos - 1))`.